

tf.compat.v1.nn.depthwise_conv2d Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/nn/depthwise_conv2d

Depthwise 2-D convolution.

Function Signatures

```
tf.compat.v1.nn.depthwise_conv2d( input, filter, strides,
padding, rate=None, name=None, data_format=None,
dilations=None )
```

Code Examples

```
tf.compat.v1.nn.depthwise_conv2d(
input,
filter,
strides,
padding,
rate=None,
name=None,
data_format=None,
dilations=None
)
```

```
tf.compat.v1.nn.depthwise_conv2d(
input,
filter,
strides,
padding,
rate=None,
name=None,
data_format=None,
dilations=None
)
```

```
output[b, i, j, k * channel_multiplier + q] = sum_{di, dj}
filter[di, dj, k, q] * input[b, strides[1] * i + rate[0] * di,
strides[2] * j + rate[1] * dj, k]
```

```
output[b, i, j, k * channel_multiplier + q] = sum_{di, dj}
filter[di, dj, k, q] * input[b, strides[1] * i + rate[0] * di,
strides[2] * j + rate[1] * dj, k]
```

```
x = np.array([
[1., 2.],
[3., 4.],
[5., 6.]
], dtype=np.float32).reshape((1, 3, 2, 1))
kernel = np.array([
[1., 2.],
[3., 4.]
], dtype=np.float32).reshape((2, 1, 1, 2))
tf.compat.v1.nn.depthwise_conv2d(x, kernel, strides=[1, 1, 1, 1],
padding='VALID').numpy()
array([[[[10., 14.],
[14., 20.]],
[[18., 26.],
[22., 32.]]]], dtype=float32)
```

```
x = np.array([
```

Documentation

Depthwise 2-D convolution.

Given a 4D input tensor ('NHWC' or 'NCHW' data formats)

and a filter tensor of shape

`[filter_height, filter_width, in_channels, channel_multiplier]`

containing `in_channels` convolutional filters of depth 1, `depthwise_conv2d` applies a different filter to each input channel (expanding from 1 channel to `channel_multiplier` channels for each), then concatenates the results together. The output has `in_channels * channel_multiplier` channels.

In detail, with the default NHWC format,

Must have `strides[0] = strides[3] = 1`. For the most common case of the same horizontal and vertical strides, `strides = [1, stride, stride, 1]`.

If any value in `rate` is greater than 1, we perform atrous depthwise convolution, in which case all values in the `strides` tensor must be equal to 1.

Usage Example:

`## Returns`